



Organizador de treinos

Documentação

Professor: *Bruno Jesus*

Alunos: *Leonardo Baron N°16,
Lucas Emanuel N°19*

Funções:

- 1- Cadastrar_treino
- 2- Consultar_treinos
- 3- Buscar_treinos
- 4- Atualizar_treinos
- 5- Remover_treino
- Exibir_menu

1- Cadastrar_treino

```
treinos = []

#cadastra as sessões
def cadastrar_treino():
    nome = str(input("👤 Entre com o nome do seu treino 👤: "))
    exercicio = str(input("🏃 Digite o exercício que será realizado 🏃: "))
    series = int(input("🔢 Digite o número de series 🔢: "))
    repeticoes = int(input("🔄 Digite o número de repetições desse exercício 🔄: "))
    dia = str(input("📅 Digite qual o dia da semana será seu treino 📅: "))

    treino = { # dicionario organiza os dados
        "nome": nome,
        "exercício": exercicio,
        "series": series,
        "repetições": repeticoes,
        "dia": dia,
        "realizado": False
    }

    treinos.append(treino)
    print("✅ Sessão cadastrada com sucesso! ✅\n")#\n pula uma linha
```

A função `cadastro_treino()` permite que o usuário registre uma sessão de treino. Ela solicita informações como nome do treino, exercício, número de séries, repetições e dia da semana. Em seguida, esses dados são organizados em um dicionário e armazenados na lista `treinos`. Por fim, o programa exibe uma mensagem confirmando que o treino foi cadastrado com sucesso.

2- Atualizar_treinos

```
def atualizar_treinos():
    consultar_treinos()
    if len(treinos) == 0:
        return
    else:
        numero = int(input('Digite o número do treino que você deseja marcar como realizada: '))
        indice = numero - 1
        if numero <= len(treinos):
            treinos[indice]['realizado'] = True
            print('✅ Treino marcado como Realizado! ✅\n')
        else:
            print('⚠️ Número inválido. ⚠️\n')
```

A função `atualizar_treinos()` permite marcar um treino como realizado. Primeiro, ela exibe a lista de treinos cadastrados e verifica se existe algum treino registrado. Em seguida, o usuário informa o número do treino que deseja atualizar. Se o número for válido, o status realizado é alterado para `True` e uma mensagem de sucesso é exibida. Caso contrário, o programa informa que o número digitado é inválido.

3- Consultar_treinos

```
def consultar_treinos():
    if len(treinos) == 0:
        print("😞 Você não possui sessões cadastradas! 😞\n")
        return #serve para sair da função
    numero = 1 # lista as sessões cadastradas
    for treino in treinos: #percorre a lista, mostrando as sessões cadastradas
        status = " " #define se esta realizada ou não
        if treino['realizado']==True: #mostra de forma visual se esta realizada ou não
            status = '✅ Realizado'
        else: #mostra de forma visual se esta realizada ou não
            status = '❌ Não Realizado'

        print(f'[{numero}] Nome: {treino['nome']} | Exercício: {treino['exercício']} | '
              f'Séries: {treino['series']} | Repetições: {treino['repetições']} | Dia: {treino['dia']} | '
              f'Status: {status}\n') # mostra as sessões cadastradas de forma visual e organizada
        numero += 1 #lista as sessões 1,2,3,4....

    print() #serve para separar as informações
```

A função `consultar_treinos()` é responsável por exibir todos os treinos cadastrados. Ela verifica se existem treinos registrados e, caso não haja, informa isso ao usuário. Se houver treinos, percorre a lista e mostra de forma organizada as informações de cada sessão, incluindo nome, exercício, séries, repetições, dia da semana e o status, indicando se o treino foi realizado ou não.

4- *Buscar_treinos*

```
def buscar_treinos():
    termo = str(input('🔍 Busque por treino ou dia da semana 📅: ')).lower()
    encontrados = [] #serve para armazenar as sessões encontradas

    for treino in treinos: #percorre a lista
        if (termo in treino['nome'].lower()) or (termo in treino['dia'].lower()): #verifica se encontra a sessão por dia ou por tema
            encontrados.append(treino) #adiciona as sessão encontradas na variavel composta encontrados

    if len(encontrados) == 0: # se não houver nenhuma sessão encontrada ele retorna o print abaixo
        print('😞 Nenhuma sessão encontrada! 😞')
        return #sai da condição

    numero = 1 # lista as sessões cadastradas
    for treino in treinos: # percorre a lista, mostrando as sessões cadastradas
        status = " " # define se esta realizada ou não
        if treino['realizado'] == True: # mostra de forma visual se esta realizada ou não
            status = '✅ Realizado'
        else: # mostra de forma visual se esta realizada ou não
            status = '❌ Não Realizado'

        print(f'[{numero}] Nome: {treino["nome"]} | Exercício: {treino["exercício"]} | '
              f'Séries: {treino["series"]} | Repetições: {treino["repetições"]} | Dia: {treino["dia"]} | '
              f'Status: {status}\n') # mostra as sessões cadastradas de forma visual e organizada
        numero += 1 # lista as sessões 1,2,3,4....
```

A função `buscar_treinos()` permite pesquisar treinos cadastrados pelo nome do treino ou pelo dia da semana. O termo informado pelo usuário é comparado com os dados armazenados, e os treinos correspondentes são adicionados a uma lista de resultados. Caso nenhum treino seja encontrado, uma mensagem é exibida informando isso. Se houver resultados, as informações dos treinos encontrados são apresentadas de forma organizada, incluindo nome, exercício, séries, repetições, dia e status de realização.

5- *Remover_treinos*

```
def remover_treinos():
    consultar_treinos()
    if len(treinos) == 0: return
    else:
        escolher = int(input('⚠ Escolha qual treino você deseja remover ⚠: '))
        num = escolher - 1
        if escolher <= len (treinos):
            treinos.pop(num)
            print('✅ Treino Removido com Sucesso! ✅\n')
        else:
            print('⚠ Número inválido. ⚠\n')
```

A função `remover_treinos()` permite excluir um treino cadastrado. Primeiro, ela exibe a lista de treinos disponíveis e verifica se existe algum treino registrado. Em seguida, o usuário informa o número do treino que deseja remover. Se o número for válido, o treino é removido da lista e uma mensagem de confirmação é exibida. Caso contrário, o programa informa que o número digitado é inválido.

Exibir_menu

```
def exibir_menu():
    while True: #mostrar as opções do sistema quantas vezes forem necessarias
        print('=== Organizador de treinos! ===')
        print('1. 📄 Cadastrar novo treino 📄')
        print('2. 📄 Consultar treinos 📄')
        print('3. 🔍 Buscar por treino ou dia 🔍')
        print('4. ✅ Marcar treino como realizado ✅')
        print('5. ❌ Remover um treino ❌')
        print('6. Sair')
        escolha = str(input('Escolha uma opção: '))
        if escolha == '1':
            cadastrar_treino()
        elif escolha == '2':
            consultar_treinos()
        elif escolha == '3':
            buscar_treinos()
        elif escolha == '4':
            atualizar_treinos()
        elif escolha == '5':
            remover_treinos()
        elif escolha == '6':
            print('👋 Saíndo do Sistema. Até a próxima! 👋')
            break
        else: # se caso o usuario digitar algo alem do 1,2,3,4,5.
            print('⚠️ Opção inválida. Tente novamente! ⚠️')
```

Esse código cria um menu interativo para um sistema de organização de treinos. A função `exibir_menu()` utiliza um laço `while True` para manter o menu aparecendo até que o usuário escolha a opção de sair. O programa exibe as opções disponíveis, recebe a escolha do usuário através do `input()` e, usando estruturas `if` e `elif`, chama a função correspondente à ação selecionada, como cadastrar, consultar, buscar, atualizar ou remover treinos. Caso o usuário digite uma opção inválida, uma mensagem de erro é exibida e o menu é mostrado novamente. Quando a opção 6 é escolhida, o programa encerra sua execução com o comando `break`.

